

PYTHON DAY 6

Lists, Collections, Mutation & Practical Data Processing

Goal: Understand lists as Python's basic mutable collection. Create, access, modify, iterate, search, sort, copy, and process groups of values.

Core mental model: A list is an ordered container holding references to objects. Today, focus on behavior first and implementation details second.

1. Warm-up: Recall Days 1–5

Write a small program using variables, input(), decisions, loops, and strings. Then ask: how would you handle 20 users without repeating the same code?

```
name = input("Name: ").strip()
age = int(input("Age: "))

if age >= 18:
    print(f"{name} is an adult.")
else:
    print(f"{name} is a minor.")
```

Challenge: Imagine collecting the names and ages of 20 people. The need to store multiple values is the problem lists solve.

2. What Is a List?

```
numbers = [10, 20, 30, 40]
names = ["Ravi", "Anu", "John"]
mixed = ["Python", 100, True, 3.14]
empty = []
```

Lists are **ordered**, **mutable**, and allow duplicate values. A list can contain different types, although real applications usually benefit from predictable structures.

Think in numbered boxes

```
scores = [85, 92, 76, 88]

# index:    0    1    2    3
# value:   85   92   76   88
```

Python uses zero-based indexing, so the first element is at index 0.

3. Indexing

```
scores = [85, 92, 76, 88]

print(scores[0])    # 85
print(scores[2])    # 76
print(scores[-1])   # 88
print(scores[-2])   # 76
```

Negative indexes count from the end. **-1** means the last element.

```
scores = [85, 92, 76]
print(scores[3])    # IndexError
```

Valid indexes range from 0 to len(scores) - 1. Boundary testing is a professional debugging habit.

4. Slicing

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])    # [20, 30, 40]
print(numbers[:3])     # [10, 20, 30]
print(numbers[2:])     # [30, 40, 50]
print(numbers[::2])    # [10, 30, 50]
print(numbers[::-1])   # reversed copy
```

The pattern is **list[start:stop:step]**. The stop position is excluded.

5. Mutation: The Big Difference

```
names = ["Ravi", "Anu", "John"]
names[1] = "Priya"
print(names)
# ['Ravi', 'Priya', 'John']
```

Lists are mutable, meaning the existing list can be changed. This is different from strings, where individual characters cannot be replaced directly.

6. Adding Items

append()

```
tasks = ["study", "gym"]
tasks.append("sleep")
print(tasks)
```

append() adds one object to the end.

extend()

```
tasks = ["study", "gym"]
tasks.extend(["code", "read"])
print(tasks)
```

extend() adds elements from another iterable.

insert()

```
names = ["Ravi", "John"]
names.insert(1, "Anu")
print(names)
```

7. Removing Items

```
items = ["pen", "book", "bag", "phone"]

items.remove("book")
print(items)

last_item = items.pop()
print(last_item)

first_item = items.pop(0)
print(first_item)

items.clear()
print(items)
```

remove(value) removes a matching value. **pop(index)** removes and returns an item. **clear()** empties the list.

Debugging task: What happens when `remove()` cannot find the value? What happens when `pop()` receives an invalid index? Test both.

8. Membership and Length

```
users = ["alice", "bob", "charlie"]

print("bob" in users)
print("admin" not in users)
print(len(users))
```

`in` and `not in` are useful for validation, filtering, and duplicate checks. `len()` gives the number of elements.

9. Iterating Through Lists

```
prices = [100, 250, 75, 500]

for price in prices:
    print(price)
```

A `for` loop naturally iterates over list elements.

Index + value with `enumerate()`

```
names = ["Ravi", "Anu", "John"]

for index, name in enumerate(names):
    print(index, name)
```

Use `enumerate()` when you need both position and value.

10. Sorting

```
numbers = [50, 10, 40, 20, 30]

numbers.sort()
print(numbers)

numbers.reverse()
print(numbers)
```

`sort()` changes the existing list. `reverse()` reverses the existing list.

`sort()` vs `sorted()`

```
numbers = [50, 10, 40]

new_numbers = sorted(numbers)

print(numbers)
print(new_numbers)
```

`sorted()` returns a new sorted list. `sort()` performs an in-place mutation and returns `None`. This distinction appears often in production code.

11. Aliasing vs Copying

```
a = [10, 20, 30]
b = a

b.append(40)
print(a)
```

```
print(b)
```

a and b refer to the same list object. Assignment does not create a new list.

```
a = [10, 20, 30]
b = a.copy()
# or: b = a[:]
```

```
b.append(40)
```

```
print(a)
print(b)
```

copy() and slicing create a separate top-level list. Nested mutable objects require deeper study later.

12. Lists and Functions

```
def add_bonus(scores):
    scores.append(100)
```

```
marks = [70, 80]
add_bonus(marks)
```

```
print(marks)
```

A function can mutate a list it receives. Understanding this prevents surprising side effects in larger programs.

13. Mini Project: Shopping List Manager

Requirements: Add an item, remove an item, view all items, search for an item, and quit. Use a while loop and a list.

```
shopping = []

while True:
    print("\n1. Add")
    print("2. Remove")
    print("3. View")
    print("4. Search")
    print("5. Quit")

    choice = input("Choose: ").strip()

    if choice == "1":
        item = input("Item: ").strip()
        shopping.append(item)
    elif choice == "2":
        item = input("Remove: ").strip()
        if item in shopping:
            shopping.remove(item)
    elif choice == "3":
        for i, item in enumerate(shopping, start=1):
            print(i, item)
    elif choice == "4":
        item = input("Search: ").strip()
        print(item in shopping)
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

Upgrade: Prevent duplicates and make searches case-insensitive.

14. Practical Project: Expense Tracker

Build a CLI expense tracker using a list of numbers.

```
expenses = []

while True:
    amount = input("Enter expense or 'done': ").strip()

    if amount.lower() == "done":
        break

    expenses.append(float(amount))

print("Expenses:", expenses)
print("Total:", sum(expenses))
print("Count:", len(expenses))

if expenses:
    print("Average:", sum(expenses) / len(expenses))
```

Upgrade: Add category and description. Later, dictionaries will give you a much cleaner representation for each expense.

15. Main Project: CLI Inventory Manager

Build a realistic small-store inventory program.

Version 1: Keep a list of product names. Add, remove, search, display with numbering, count products, sort alphabetically, and exit through a menu.

Version 2: Represent each product as [name, quantity, price] and calculate total inventory value. This deliberately creates a design problem that dictionaries will solve later.

Version 3: Add validation, duplicate handling, and low-stock warnings.

16. Debugging Lab

Predict the output before running each example. Then explain why.

```
# A
items = ["a", "b", "c"]
items.append(["d", "e"])
print(items)

# B
items = ["a", "b", "c"]
items.extend(["d", "e"])
print(items)

# C
a = [1, 2]
b = a
b.append(3)
print(a)

# D
a = [1, 2]
b = a.copy()
b.append(3)
print(a)
```

```
# E
numbers = [3, 1, 2]
result = numbers.sort()
print(result)
print(numbers)
```

17. Research Experiment: What Is a List Really?

```
items = [10, 20, 30]

print(type(items))
print(id(items))

items.append(40)

print(id(items))
print(items)
```

Investigate: Does `append()` replace the list object? What does mutability mean? What does a variable reference? Why can one list contain different object types? Read the official Python documentation and compare it with your experiment.

18. Knowledge Check

Question	Expected understanding
Why does indexing start at 0?	Positional indexing and offsets.
<code>append()</code> vs <code>extend()</code> ?	One object vs elements from an iterable.
What is mutation?	Changing an existing object.
<code>sort()</code> vs <code>sorted()</code> ?	In-place mutation vs a new sorted result.
Why did <code>b = a</code> affect <code>a</code> ?	Both names referenced the same list.
How do you make a simple copy?	<code>copy()</code> or slicing.
What does <code>pop()</code> return?	The removed element.
How do you search?	Use <code>in</code> / not <code>in</code> .

19. Completion Checklist

- I can create lists and access elements by index.
- I understand positive and negative indexing.
- I can slice lists.
- I understand list mutability.
- I can use `append()`, `extend()`, `insert()`, `remove()`, `pop()`, and `clear()`.
- I can use `in` and `not in`.
- I can iterate through a list.
- I understand `enumerate()`.
- I know `sort()` vs `sorted()`.
- I understand aliasing and basic copying.

- I completed the Shopping List Manager.
- I completed the Expense Tracker.
- I attempted the CLI Inventory Manager.
- I explained every debugging experiment.

20. Recommended Day 6 Schedule

30 min: Read and type every example yourself.

45 min: Practice indexing, slicing, mutation, and methods.

45 min: Shopping List Manager.

60 min: Expense Tracker.

60–90 min: Inventory Manager.

20 min: Debugging + research experiment.

21. Official References

Python Tutorial: Data Structures

Python Library Reference: Built-in Types

Built-in Functions: `len()`, `enumerate()`, `sorted()`

22. Day 7 Preview

Dictionaries, Tuples, Sets & Choosing the Right Collection

Next you will move from simple groups of values to structured data. You will learn key-value storage, uniqueness, set operations, tuple immutability, and how to choose between list, tuple, set, and dictionary.